Note :

Primitive Variables : Will always hold the value

Reference Variables : Will hold the address.

HEAP & STACK Diagram for Sample.java

HEAP MEMORY

1000x : SampleObject, i1 : 900

2000x : SampleObject, i1 : ~~900~~ 9

3000x : SampleObject, i1 : ~~900~~ 20

Output
20
9

STACK MEMORY

main_stack
----
s1 : ~~1000x~~ null
s2 : 2000x
s3 : null

modify_stack
----
s : ~~2000x~~ ~~3000x~~ null

```
public class Sample
{
    private int i1 = 900;

    void main()
    {
        Sample s1 = new Sample();

        Sample s2 = new Sample();

      Sample s3 = modify(s2);

        s1 = null;

     //GC [2 objects are eligible for GC 1000x, 3000x]

        IO.println(s2.i1);
    }

   public Sample modify(Sample s) //Method return type as a class
    {
        s.i1=9;
        s  = new Sample();
        s.i1= 20;
     IO.println(s.i1);
        s=null;
        return s;
    }
}
```

HEAP & STACK Diagram for Employee.java

HEAP MEMORY

1000x : EmployeeObject, id : ~~100~~ ~~600~~ 200

2000x : EmployeeObject, id : ~~100~~ 400

3000x : EmployeeObject. id : ~~100~~ ~~350~~ 200

4000x : EmployeeObject, id : ~~100~~ 350

Output :

400
200
200
200

STACK MEMORY

main_stack
----
val : 600
e1 : 1000x
e2 : 3000x

update_stack
----
e : ~~1000x~~ 2000x

switchEmployees_stack
----
e1 : 3000x
e2 : ~~1000x~~ 4000x
temp : 350

```
public class Employee
{
    int id = 100;

    void main()
    {
        int val = 600;

        Employee e1 = new Employee();

        e1.id = val;

        update(e1);

        IO.println(e1.id);

     Employee e2 = new Employee();

        e2.id = 350;

        switchEmployees(e2,e1);  //3000x, 1000x

         //GC [2 objects 2000x & 4000x are eligible for GC]

            IO.println(e1.id);
          IO.println(e2.id);
      }

     public  void update(Employee e)
     {
     e.id = 200;
        e = new Employee();
        e.id = 400;
        IO.println(e.id);

     }

     public  void switchEmployees(Employee e1, Employee e2)
      {
         int temp = e1.id;
         e1.id = e2.id; //200
         e2 = new Employee();
         e2.id = temp;
      }
  }
```

Passing an object reference to the Constructor (Copy Constructor)
----

* We can pass an object reference to the constructor, the main purpose of passing the object reference to the constructor, **to copy the content of one object to another object**.
[We can copy one object data to another object]

Example :

```
public class Manager
{
  private int managerId;
  private String managerName;
  private double managerSalary;

  public Manager(Employee emp) //emp = e1   [Copy Constructor]
  {
      this.managerId = emp.getEmployeeId();
      this.managerName = emp.getEmployeeName();
      this.managerSalary = emp.getEmployeeSalary() + 50000;
  }
}
```

//Program on Copy constructor :

```
package com.ravi.copy_constructor;

public class Employee
{
    private int employeeId;
    private String employeeName;
    private double employeeSalary;

    public Employee(int employeeId, String employeeName, double employeeSalary)
    {
        super();
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.employeeSalary = employeeSalary;
    }


    public int getEmployeeId() {
        return employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public double getEmployeeSalary() {
        return employeeSalary;
    }


    @Override
    public String toString()
    {
        return "Employee [employeeId=" + employeeId + ", employeeName=" + employeeName + ",
employeeSalary="
                + employeeSalary + "]";
    }

}


package com.ravi.copy_constructor;

public class Manager
{
  private int managerId;
  private String managerName;
  private double managerSalary;

  public Manager(Employee emp) //emp = e1
  {
      this.managerId = emp.getEmployeeId();
      this.managerName = emp.getEmployeeName();
      this.managerSalary = emp.getEmployeeSalary() + 50000;
  }

  @Override
  public String toString()
  {
    return "Manager [managerId=" + managerId + ", managerName=" + managerName + ",
managerSalary=" + managerSalary
             + "]";
  }

}


package com.ravi.copy_constructor;

public class CopyConstructorDemo1 {

    public static void main(String[] args)
    {
        Employee e1 = new Employee(111, "Scott", 90000);
        IO.println(e1);

        Manager m1 = new Manager(e1);
        IO.println(m1);

    }

}
```

Note : In the above program to initialize the non static fields of Manager class, we are using Employee class data.

The following program explains, how to copy the same class object data to another object.

```
package com.ravi.copy_constructor;

public class ProductDemo {

    public static void main(String[] args) {

        Product p1 = new Product(111, "Camera", 45000);
        IO.println("First Object "+p1);

        Product p2 = new Product(p1);
        IO.println("Second Object "+p2);
    }
}

class Product
{
    private int id;
    private String name;
    private double price;


    public Product(int id, String name, double price) {
        super();
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Product(Product prod)  //prod = p1
    {
        this.id = prod.id;
        this.name = prod.name;
        this.price = prod.price;
    }

    @Override
    public String toString()
    {
        return "Product [id=" + id + ", name=" + name + ", price=" + price + "]";
    }

}
```

Product Object (1000x) — p1, prod
```
id = 111
name = Camera
price = 45000
```

product Object 2000x — p2
```
id = prod.id;
name = prod.name;
price = prod.price;
```